A PROJECT REPORT ON

Zero-Shot Text Classification Using Natural Language Inference

Natural Language Processing

Assignment 1

Topic 3: Zero-Shot Classification

Chapter 2: Application Topics

Abstract

Text classification is commonly implemented using supervised models trained on a fixed set of categories. Such models require labelled examples and may need further training when the category set changes. This project examines an alternative approach in which categories are described in natural language and evaluated using a pretrained natural language inference model. A short-text classifier is designed around the BART-MNLI checkpoint, with news categorization used as the application context. Each input is paired with a hypothesis constructed from a candidate label, and the labels are ranked using the model's entailment scores. The design includes input validation, hypothesis construction, tokenization, inference and result interpretation. A Python implementation is presented using the Hugging Face Transformers library. The evaluation framework considers classification accuracy, macro-averaged F1 score, ambiguity and sensitivity to category descriptions. The approach permits changes to the application label set without retraining the model, but its reliability depends on the quality of label descriptions and the relationship between the target domain and the model's prior training. The project establishes a practical baseline for text categorization where task-specific training data are limited.

Keywords: zero-shot classification; natural language inference; BART; text categorization

Index

Abstract 2

1. Introduction 4

2. Problem Statement 5

3. Design 6

4. Proposed Methodology 7

5. Implementation 9

6. Result 11

7. Reference 12

1. Introduction

The increasing volume of textual information in news services, educational platforms and digital communication systems creates a need for automatic categorization. Assigning a document to a suitable topic makes information easier to retrieve and supports downstream processes such as routing, filtering and analysis. In conventional supervised text classification, a model learns a mapping from text to a predefined label set using labelled examples. This approach is effective when representative training data are available, but preparing such data can be expensive.

Zero-shot text classification addresses a setting in which labelled examples are unavailable for the target classification task. Instead of treating a category as an arbitrary identifier, the system uses the linguistic meaning of its description. Yin, Hay and Roth formulate zero-shot classification as textual entailment, allowing a model trained on an inference task to evaluate labels supplied at prediction time [1]. The term zero-shot refers to the absence of target-task training examples; the underlying language model has already undergone pretraining and task adaptation.

1.1 Natural language inference

Natural language inference (NLI) evaluates the relationship between a premise and a hypothesis. The principal relations are entailment, contradiction and neutrality. Entailment indicates that the premise supports the hypothesis; contradiction indicates incompatibility; neutrality indicates that support cannot be established. The MultiNLI corpus provides sentence pairs from several genres for studying these relationships [3]. In classification, a candidate label is expressed as a hypothesis, such as "This text is about sports," and compared with the input sentence.

1.2 Model selection and application

BART is a Transformer-based encoder-decoder model developed through denoising pretraining [2]. The facebook/bart-large-mnli checkpoint adapts this model to NLI [4]. It is selected here to construct a news classifier using sports, technology, politics and education as candidate topics. These categories form the initial application configuration rather than a new training dataset. The same implementation can evaluate another coherent topic set, provided the new setting is assessed independently.

2. Problem Statement

The problem is to assign a short English text to the most relevant category from a user-supplied label set without training a dedicated classifier on examples from those categories. The system must preserve the meaning of the input, compare all candidates consistently and return an interpretable ranking. A fixed keyword rule is insufficient because relevant information may be expressed through synonyms, paraphrases or indirect descriptions.

2.1 Objectives

The objectives are to develop a reusable classification procedure, represent categories as natural-language hypotheses, obtain ranked scores from a pretrained NLI model and define an evaluation protocol suitable for the intended domain. The implementation must also reject empty inputs and duplicate categories, since invalid application data can make otherwise valid model output misleading.

2.2 Scope and constraints

The proposed system processes short English passages and initially assumes that one topic is to be selected. Candidate labels must describe comparable concepts. For example, sports and politics are comparable topics, whereas sports and positive describe different aspects of a text. Long documents require explicit segmentation or a documented truncation policy because the model has a finite input window. Performance on other languages, specialized terminology and multi-topic documents is outside the initial scope.

Table 1. Functional requirements of the proposed classifier.

Requirement	Specification
Input	A non-empty text and at least two unique topic labels
Processing	One premise-hypothesis comparison for each candidate
Output	Ranked labels and corresponding model scores
Learning setting	No fine-tuning on the target application labels
Execution	Local Python inference using a pretrained checkpoint

3. Design

3.1 System architecture

The architecture separates application input from model inference. The input module receives the passage and candidate categories. Validation establishes that both are usable before hypotheses are constructed. The tokenizer converts each text-hypothesis pair into the representation expected by the checkpoint. The inference module obtains NLI outputs, and post-processing produces a category ranking. The output module associates each score with its label and retains the original passage.

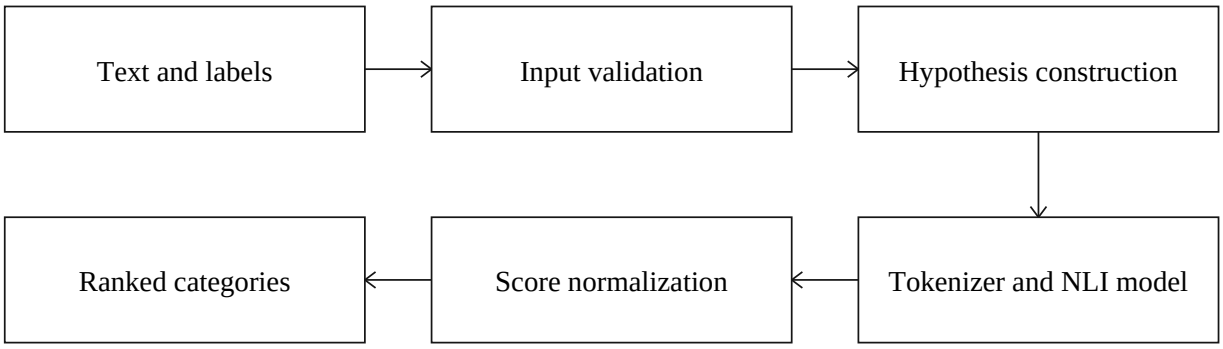


Figure 1. Processing architecture for zero-shot text classification.

3.2 Representation and output

For an input x and label c_i, the hypothesis is h_i = "This text is about c_i." The resulting pair is supplied to the model without removing punctuation or negation. The output is represented as a dictionary containing the original sequence, labels sorted by score and the corresponding score list. These lists must be interpreted together; sorting scores separately would destroy the association between a category and its value.

3.3 Design rationale

The model is loaded once and reused, avoiding repeated initialization for every passage. Label descriptions and the hypothesis template are held constant during evaluation to ensure that changes in results can be attributed to the input rather than to inconsistent configuration. A local command-line implementation keeps the core method independent of a user interface. An interface can subsequently call the same inference procedure without altering its scoring logic.

4. Proposed Methodology

4.1 Data preparation and hypothesis construction

The procedure begins with a short passage and a coherent set of candidate labels. Leading and trailing whitespace is removed; stop-word removal and stemming are not applied. Those operations are unnecessary for this pretrained pipeline and may alter grammatical relationships. Each label is inserted into a fixed hypothesis template. For the input "The team won the final cricket match," the sports hypothesis is supported by the subject matter, while hypotheses about politics or technology are less relevant.

4.2 Inference and single-label scoring

Each premise-hypothesis pair is evaluated using the selected NLI checkpoint. Let e_i denote the entailment logit for candidate i and let K be the number of candidates. For the single-label configuration with at least two candidates, the pipeline applies a softmax over the candidate entailment logits [6]. The normalized score and predicted category are:

$$s_i = exp(e_i) / \Sigma_{j=1}^{K} exp(e_j) \quad (1)$$

$$\hat{c} = arg\ max_{c_i}\ s_i \quad (2)$$

The scores sum to one across the supplied candidate set. They express relative preference among those candidates, so modifying the label set can change the scores even when the passage remains identical. A high score is therefore not a calibrated guarantee that a category is correct. The single-label procedure also returns a top candidate when all supplied categories are unsuitable.

4.3 Algorithm

Step 1: Read and validate the text and candidate labels.
Step 2: Construct a hypothesis for every candidate.
Step 3: Encode each premise-hypothesis pair using the model tokenizer.
Step 4: Obtain the NLI output and extract candidate entailment logits.
Step 5: Normalize scores across labels and sort them in descending order.
Step 6: Return the ranked labels, scores and original input.

4.4 Evaluation procedure

A balanced evaluation set can contain 40 independently labelled news sentences, with 10 assigned to each of the four topics. The label set and hypothesis template should be fixed before final evaluation. Sentences used to select the template must be excluded from the final set. Accuracy measures the proportion of correct top-ranked predictions. Precision and recall are calculated for each class, and macro-F1 averages the class-specific F1 values so that all classes contribute equally.

$$\textit{Accuracy} = \textit{Number of correct predictions} / \textit{Number of test examples} \tag{3}$$

A confusion matrix identifies pairs of topics that are repeatedly confused. Additional cases should include negation, paraphrases, unfamiliar entities and mixed-topic passages. Runtime measurements should distinguish the initial model download and loading time from inference time. Performance must be reported together with hardware, software versions and the checkpoint revision.

5. Implementation

5.1 Software environment

The implementation uses Python, PyTorch and the Hugging Face Transformers library. The pipeline interface handles tokenization, model execution and post-processing [5]. The task name and checkpoint are specified explicitly to make the implementation reproducible. The device value -1 selects CPU execution. A compatible GPU may be used for a larger workload, but its benefit should be measured rather than assumed.

Installation: `python -m pip install torch transformers`
Save Listing 1 as `zero_shot.py` and execute it using `python zero_shot.py`. The first run downloads the model files.

Listing 1. Core implementation of the zero-shot classifier.

```
from transformers import pipeline

classifier = pipeline(
    "zero-shot-classification",
    model="facebook/bart-large-mnli",
    device=-1
)

text = "The team won the final cricket match."
labels = ["sports", "technology", "politics", "education"]

if not text.strip() or len(labels) < 2:
    raise ValueError("Text and at least two labels are required.")
if any(not label.strip() for label in labels):
    raise ValueError("Candidate labels cannot be empty.")
if len(set(label.casefold() for label in labels)) != len(labels):
    raise ValueError("Candidate labels must be unique.")

result = classifier(
    text,
    candidate_labels=labels,
    hypothesis_template="This text is about {}.",
    multi_label=False
)

for label, score in zip(result["labels"], result["scores"]):
    print(f"{label:12s} {score:.4f}")
```

5.2 Implementation details

The passage and labels are supplied independently. Validation rejects blank text, fewer than two labels, empty category names and duplicate labels. The hypothesis template expresses the intended classification aspect consistently. Setting multi_label=False requests the single-label normalization described in Equation (1). The returned label-score pairs are printed together in their ranked order. Decimal formatting affects presentation only; it does not change the model's internal scores.

In a batch application, the model object should be retained between calls. Inputs should be checked against the tokenizer's sequence limit before inference. Long inputs can be segmented into passages, but a rule for combining passage-level predictions must be specified and evaluated. Download failures, unavailable model files and insufficient memory should be handled as execution errors rather than interpreted as classification outcomes.

6. Result

6.1 Qualitative case analysis

Table 2 gives illustrative reference categories, not measured outputs. Separately, real inference classified all twelve demonstration sentences correctly (macro-F1 1.00). Predictions and checkpoint details are in the repository. This small set is not a benchmark.

Table 2. Illustrative test cases for topic classification.

Test passage	Reference category	Basis for interpretation
The team won the final cricket match.	Sports	Match and team context
A new smartphone chip improves battery life.	Technology	Electronic device and processor
Parliament debated the proposed legislation.	Politics	Government and lawmaking
Students attended a mathematics workshop.	Education	Learning activity and subject

6.2 Interpretation and limitations

Straightforward topic statements provide an initial check of the application workflow, but they are insufficient to establish general performance. A passage about funding for sports education may legitimately relate to sports, politics and education. Its classification depends on the intended labelling policy. A consistent policy is required before such cases are included in an accuracy calculation.

Zero-shot classification also depends on label wording. A descriptive label may be easier to interpret than an abbreviation, and overlapping labels may divide the evidence. A label set that omits the correct category can still produce a confident-looking ranking because the scores are normalized over available alternatives. Human review is appropriate for ambiguous cases, while any automatic rejection threshold should be selected on a separate validation set.

6.3 Conclusion

The proposed method provides a practical way to categorize short texts when target-task training examples are unavailable. Its principal advantage is that meaningful label descriptions can be changed without updating model weights. The architecture and implementation establish the classification workflow, while the evaluation protocol defines how its usefulness should be measured. Future development may include multi-label classification, domain-specific label descriptions and a comparison with a supervised baseline trained on a small labelled dataset.

7. Reference

[1] W. Yin, J. Hay, and D. Roth, "Benchmarking Zero-shot Text Classification: Datasets, Evaluation and Entailment Approach," in *Proceedings of EMNLP-IJCNLP*, pp. 3914-3923, 2019. https://aclanthology.org/D19-1404/.

[2] M. Lewis et al., "BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension," in *Proceedings of ACL*, pp. 7871-7880, 2020. https://aclanthology.org/2020.acl-main.703/.

[3] A. Williams, N. Nangia, and S. R. Bowman, "A Broad-Coverage Challenge Corpus for Sentence Understanding through Inference," in *Proceedings of NAACL-HLT*, pp. 1112-1122, 2018. https://aclanthology.org/N18-1101/.

[4] Hugging Face, "facebook/bart-large-mnli," model card. Accessed: October 12, 2026. https://huggingface.co/facebook/bart-large-mnli.

[5] Hugging Face, "Pipelines," *Transformers Documentation*. Accessed: October 12, 2026. https://huggingface.co/docs/transformers/main/en/main_classes/pipelines.

[6] Hugging Face, "ZeroShotClassificationPipeline," *Transformers source code*. Accessed: October 12, 2026. https://github.com/huggingface/transformers/blob/main/src/transformers/pipelines/zero_shot_classification.py.

Project implementation

Source code, setup instructions and tests:
https://github.com/Katori-bo/zero-shot-news-classifier